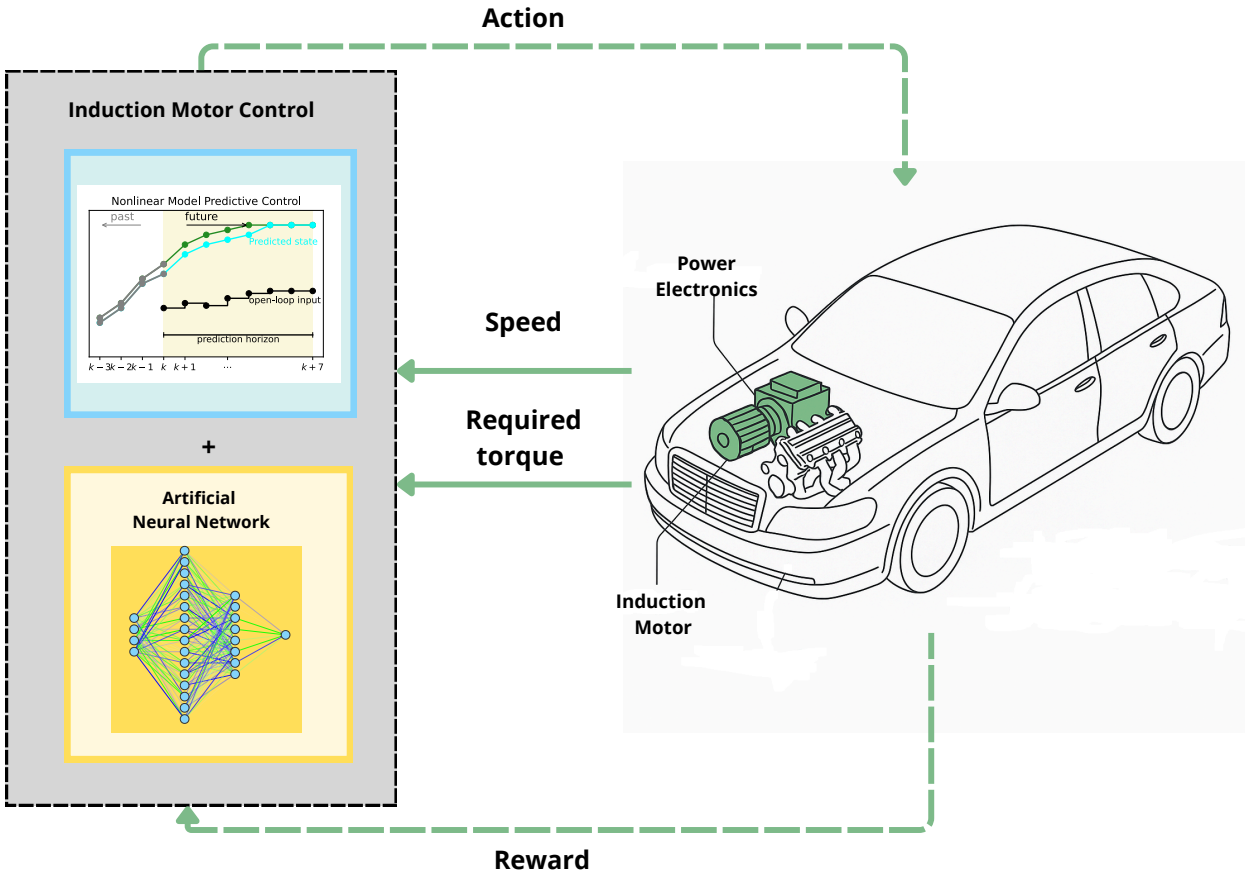
Graphical Abstract

A Hierarchical Reinforcement Learning Framework for Train Traction and Brake Coordination Using Hybrid SAC

Leonardo Tadeu Lima Franco, Matheus Valente Lopes, Kevin Mauricio Menon Ribeiro, Jony J. Eckert, Auteliano A. Santos



Highlights

A Hierarchical Reinforcement Learning Framework for Train Traction and Brake Coordination Using Hybrid SAC

Leonardo Tadeu Lima Franco, Matheus Valente Lopes, Kevin Mauricio Menon Ribeiro, Jony J. Eckert, Auteliano A. Santos

- 85 caracteres com espaços
- 85 caracteres com espaços
- 85 caracteres com espaços
- 85 caracteres com espaços
- 85 caracteres com espaços

A Hierarchical Reinforcement Learning Framework for Train Traction and Brake Coordination Using Hybrid SAC

Leonardo Tadeu Lima Franco [a,*], Matheus Valente Lopes [a], Kevin Mauricio Menon Ribeiro [a], Jony J. Eckert [a], Auteliano A. Santos [a]

[a] *School of Mechanical Engineering - University of Campinas - UNICAMP, Campinas, SP, Brazil*

Abstract

Efficient and safe train operation depends on the precise coordination of traction and braking systems under diverse track and environmental conditions. This paper presents a hierarchical reinforcement learning framework based on the Hybrid Soft Actor-Critic (SAC) algorithm for optimal train control. The proposed architecture is composed of three agents: a high-level hybrid SAC agent that decides whether the train should accelerate or brake, and two low-level hybrid SAC agents that determine the magnitude of acceleration or braking. All agents are trained within a simulated environment developed in SIMULINK. Unlike previous studies, the proposed system directly controls the locomotive's throttle point and the brake pipe pressure. The main objective of this work is to design a safe and efficient control system capable of deciding both when and how much to accelerate or brake according to the surrounding conditions. The results demonstrate the potential of hierarchical reinforcement learning to enable adaptive and reliable train operation in complex railway networks.

Keywords: Hierarchical reinforcement learning, braking and throttle control, nonlinear control, hybrid SAC, train control optimization

1. Introduction

Heavy haul railways play an important role in economic development by providing an efficient and reliable means of transporting large volumes of heavy and bulk commodities. They are essential for moving minerals, metals, and other raw materials, particularly in resource-rich countries, connecting these products to global markets. By carrying goods in large quantities, heavy haul trains reduce the transportation cost per ton and improve industrial competitiveness.

In addition, this transport mode reduces the burden on road infrastructure, helping to decrease congestion and infrastructure wear. Investments in heavy haul railway systems also create jobs, support the development of remote regions, and encourage technological progress in related areas. As a more sustainable alternative to road transport, heavy haul railways contribute to lower emissions and energy consumption, linking economic growth with environmental responsibility.

Proper braking operation is essential for the safety and efficiency of heavy haul trains. Since these trains transport very large loads over long distances, an adequate braking strategy is required to maintain operational control, especially along track sections with different gradients and terrain conditions. Correct brake application provides more stable deceleration and stopping, reducing the risk of derailment and helping to limit wear on railway infrastructure. Smooth braking can also improve energy use and extend the service life of braking components. In heavy haul operations, where the transported cargo may include hazardous materials, braking reliability is particularly important because it helps protect people, equipment, and the environment. Therefore, the proper use of braking systems is a key aspect of the operational integrity of heavy haul railways.

Pneumatically controlled brake systems remain widely used in heavy haul railway operations in several countries (Wei et al., 2017; Eckert et al., 2023). However, one of the main limitations of this type of system is the delay in brake command propagation along long train composition, since the application signal may take several seconds to reach the rear wagons. This delay can intensify longitudinal impacts at the couplers, as rear freight cars may continue pushing the vehicles ahead before the braking effort is fully developed throughout the train (Oprea et al., 2013). A possible alternative to mitigate this limitation is the use of electro-pneumatic valves, which are electronically actuated and allow brake commands to be applied almost simultaneously along the entire consist. In addition, electro-pneumatic systems enable more flexible brake cylinder pressure modulation, including more frequent pressure increases and partial releases, since the brake pipe is no longer used as the direct control medium and can continuously supply air to the freight car reservoirs. Therefore, beyond

*R

Email addresses: 1239274@dac.unicamp.br (Leonardo Tadeu Lima Franco), XXX (Matheus Valente Lopes), XXX (Kevin Mauricio Menon Ribeiro), javorski@fem.unicamp.br (Jony J. Eckert), aute@fem.unicamp.br (Auteliano A. Santos)

reducing the delay associated with conventional pneumatic actuation, electro-pneumatic braking creates opportunities for more advanced control strategies. This is particularly relevant for heavy haul train operation, where braking decisions are strongly coupled with traction and dynamic braking actions, and where inappropriate transitions between these modes may affect the longitudinal dynamic response of the train.

This work aims to develop an integrated driving control strategy for heavy freight trains, considering acceleration and braking processes together. The optimal operation of this type of train has been extensively investigated in the literature, especially in relation to energy efficiency, trajectory optimization, smooth driving, operation on downhill sections, and reduction of longitudinal forces between vehicles (Lin et al., 2017; Howlett, 2016; Bai et al., 2020; Wang et al., 2019; Zhang & Zhuan, 2013; Yi et al., 2022). In this context, Eckert et al. (2025) specifically advanced the optimization of the electro-pneumatic braking process in heavy freight trains, using multi-objective fuzzy control to reduce the energy dissipated during braking and the force peaks in the shock and traction devices. The results obtained by these authors highlighted the potential of optimized control strategies to improve the dynamic performance of the train. However, the approach remained restricted to the braking subsystem. Thus, this work starts from this basis and expands the control problem by including, in the same decision structure, the coordination between traction, dynamic braking, and pneumatic braking. To this end, an approach based on the Hybrid Soft Actor-Critic (SAC) model is adopted, as described by Delalleau et al. (2019), allowing the representation of hybrid decisions in which the choice of actuation mode and the intensity of the applied command are treated jointly.

The integrated formulation of railway operation is directly related to recent advances in automatic train operation systems, known as Automatic Train Operation (ATO). In these systems, traction and braking controls are typically adjusted in a coordinated manner to meet different operational objectives, such as energy efficiency, timetable adherence, comfort, safety, and stopping precision (Zhou et al., 2020; Chen et al., 2023; Huang et al., 2025).

In recent years, machine learning-based methods have been frequently explored in automated train operation systems, as they allow for adjusting traction and braking commands based on interaction with the simulation environment, reducing dependence on predefined speed profiles. Zhou et al. (2020) proposed intelligent railway operation algorithms based on expert knowledge and reinforcement learning, using Deep Deterministic Policy Gradient (DDPG) and Normalized Advantage Function (NAF), two approaches suitable for continuous control problems, to adjust traction and braking commands in metro systems. Similarly, Huang et al. (2025) combined an expert system and deep learning through reinforcement for the continuous control of traction and braking forces, considering criteria such as safety, comfort, punctuality, stopping precision, and energy efficiency.

Other recent formulations have also explored different ways of representing the railway control problem. Some studies use Deep Q-Network (DQN), a value-based learning method, to select discrete driving actions, while others employ Proximal Policy Optimization (PPO), a deep reinforcement learning algorithm used in control problems with safety constraints (Chen et al., 2023; Cheng et al., 2024). There are also approaches that incorporate reference systems, in which the controller learns to follow or correct a desired trajectory, and studies focused on discrete traction and braking controls, closer to the real operating interfaces of certain trains (Shang et al., 2021; Wei et al., 2022). Despite these advances, most applications are still concentrated in subways, urban railways, and high-speed trains, whose operating conditions are more controlled and whose train compositions are significantly smaller than those seen in heavy freight trains.

This difference in application is relevant because operating heavy freight trains imposes more severe dynamic constraints. The train's great length, the high mass transported, the delays associated with the pneumatic system, and the occurrence of significant longitudinal forces make the control problem more sensitive to transitions between traction, relief, dynamic braking, and pneumatic braking (Eckert et al., 2025; Wu et al., 2023). Therefore, strategies developed for urban systems should be carefully evaluated before being applied to heavy-duty train compositions. In this type of operation, a seemingly adequate control system in terms of operational performance can generate undesirable dynamic responses along the train, especially in the shock and traction devices, whose modeling directly influences the estimation of longitudinal forces on the train composition (Eckert et al., 2025; Wu et al., 2018; Wagner et al., 2021; Yadav & Vyas, 2023; Wu et al., 2014).

Therefore, detailed modeling of the longitudinal dynamics becomes essential to evaluate the performance of the proposed controller. Adequate representation of the shock and tension devices is important because different formulations of this component can significantly alter the estimation of internal forces in the composition (Wu et al., 2018; Wagner et al., 2021; Yadav & Vyas, 2023; Wu et al., 2014). Similarly, modeling the pneumatic brake system is fundamental to capturing the effects of pressure propagation and its influence on the longitudinal response of the train (Wu et al., 2023). Eckert et al. (2025) showed that simply replacing conventional pneumatic valves with electro-pneumatic valves, without a proper change in the control strategy, does not guarantee significant improvements in the dynamic behavior of the train. This result reinforces the need to develop controllers capable of exploiting the benefits of electro-pneumatic systems in a coordinated manner with the other operating modes of the train.

Based on the above, this work proposes a hierarchical learning architecture based on SAC agents for the integrated control of heavy freight train operation. Instead of treating acceleration and braking as independent problems, the proposed approach organizes driving decisions into a common control framework. The proposed structure is organized into two levels: a main agent selects the mode of action to be applied, while subordinate agents determine the intensity

of the corresponding command. In this way, the different operating modes available in driving, such as traction, dynamic braking, and pneumatic braking, are addressed within the same decision structure. This organization aims to make the control problem more structured, allowing the decision about the operating mode to be separated from the definition of the magnitude of the applied command.

Within this framework, the proposed strategy connects the choice of driving mode to the continuous definition of the commands applied to the train. The main contribution of this work, therefore, lies in the development of an integrated control strategy for heavy freight trains, combining hierarchical learning based on SAC agents with detailed dynamic models of the train composition. Unlike approaches focused solely on optimizing braking or trajectory (Bai et al., 2020; Eckert et al., 2025), the proposed controller dynamically adjusts the available operating modes while driving. The proposed approach is evaluated based on the dynamic response of the train composition, simultaneously considering the effects of traction, dynamic braking, and pneumatic braking on longitudinal forces along the train. This evaluation depends on the adequate representation of longitudinal dynamics and the shock and traction devices. Thus, this work advances beyond approaches focused solely on trajectory or braking optimization, treating train operation as an integrated decision and control problem.

The rest of this work is divided as follows: Section 2 presents the train longitudinal dynamics modeling approach, including the detailed model of the pneumatic braking system and the friction-based draft gear model. Section 3 describes the control problem formulation and the proposed hierarchical architecture based on Soft Actor-Critic agents. Section 4 presents the training, validation, and comparison results of the controller under different operational scenarios. Finally, Section 5 summarizes the main conclusions of the work and discusses possible directions for future research.

2. Train Longitudinal Dynamics

The simulation framework adopted in this study follows the train longitudinal dynamics formulation described by Cole et al. (2006). In this approach, the motion of each vehicle is obtained from the longitudinal force balance, allowing the instantaneous acceleration $\ddot{x}_n$ [m/s^2] to be evaluated as a function of the corresponding vehicle mass m_n [kg]. The train consist analyzed here comprises two Dash9 locomotives placed at the head end, followed by 168 ore freight cars, as illustrated in Fig. 1.

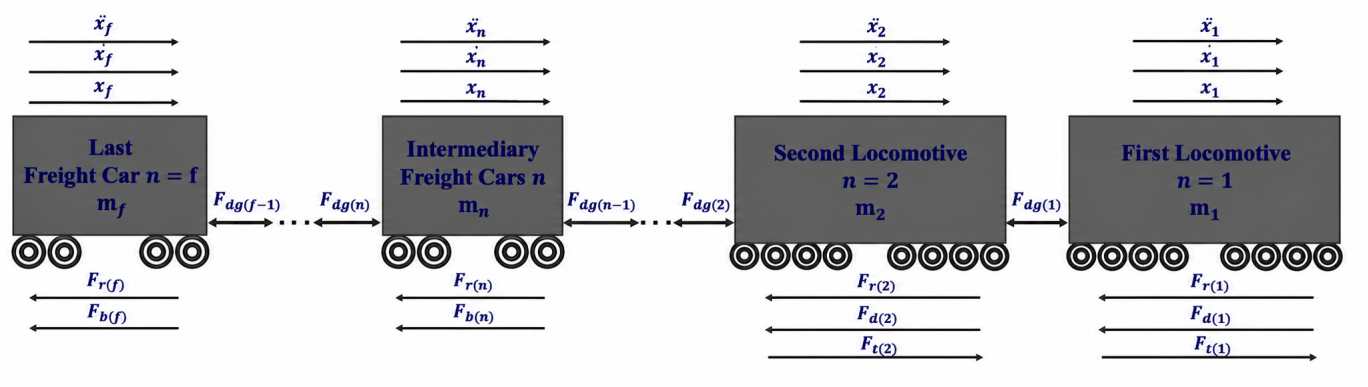


Figure 1: Kinematic representation of the longitudinal train model

Each vehicle of the train is subjected to a motion resistance force, as defined in Eq. 1. This force accounts for the combined effects of wheel-rail rolling resistance and aerodynamic drag, which are evaluated using the empirical coefficients listed in Table 1. The influence of track geometry is also included in the model by considering the local track gradient, represented by α_n [rad], and the curve radius R_n [m].

$$F_{r(n)} = m_n g \left(A_{1(n)} + \frac{A_{2(n)} N_{a(n)}}{m_n} + B_n \dot{x}_n + C_n \dot{x}_n^2 + sin\alpha_n + \frac{6.116}{g R_n} \right). \tag{1}$$

The longitudinal accelerations of the two head-end locomotives are obtained from the dynamic equilibrium of each vehicle in the longitudinal direction, as expressed by Eqs. 2 and 3.

$$\ddot{x}_1 = \frac{F_{t1} - F_{d1} - F_{r1} - F_{dg1}}{m_1}, \tag{2}$$

$$\ddot{x}_2 = \frac{F_{t2} - F_{d2} - F_{r2} + F_{dg1} - F_{dg2}}{m_2}, \tag{3}$$

where m_1 and m_2 are the masses of the first and second locomotives, respectively, and $\ddot{x}_1$ and $\ddot{x}_2$ are their corresponding longitudinal accelerations. The terms F_{t1} and F_{t2} denote the traction forces applied to the locomotives, while F_{d1} and F_{d2} represent the dynamic braking forces. The terms F_{r1} and F_{r2} are the motion resistance forces defined by Eq. 1. The quantities F_{dg1} and F_{dg2} denote the draft-gear forces transmitted through the coupler connections between adjacent vehicles.

The traction and dynamic braking forces are defined according to the train control schedule, which specifies the operating condition of each Dash9 locomotive. In traction mode, the force $F_{t(n)}$ [N] is obtained from the tractive-effort curves associated with the eight power notches, from P_1 to P_8, as shown in Fig. 2a. In dynamic braking mode, the force $F_{d(n)}$ [N] follows the four braking stages, from D_1 to D_4, as illustrated in Fig. 2b. After these locomotive forces

Table 1: Locomotives and freight cars parameter

Longitudinal train dynamics parameters				
Parameters	**Locomotives**		**Freight cars**	
	$\dot{x_n} \leq 0.1$	$\dot{x_n} > 0.1$	$\dot{x_n} \leq 0.1$	$\dot{x_n} > 0.1$
A_1	5	0.1	15	0.15
A_2	425	8.5	425	4.25
B	0	0.00938	0	0.007
C	0	0.0046	0	0.000325
Mass m_n	162×10^3 kg		106.5×10^3 kg	
Number of axis N_a	5		4	

are defined, the resulting acceleration also depends on the draft gear forces $F_{dg(n)}$ [N], which represent the interaction between adjacent vehicles and are discussed in Section 2.1.

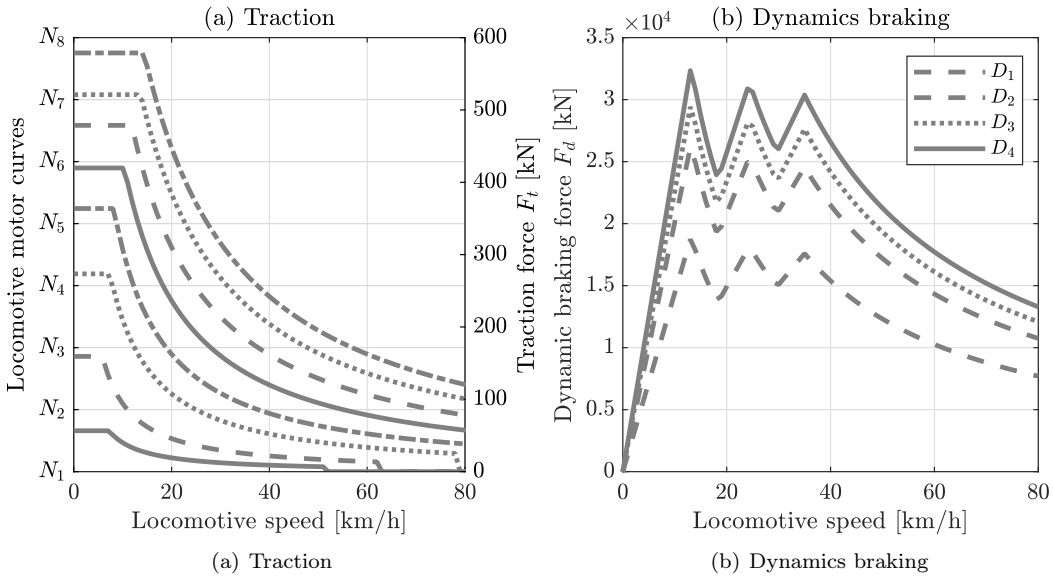


Figure 2: Locomotive traction and dynamic brake curves refiz a figura numa escala de cinza

The acceleration of the intermediate freight cars, $\ddot{x}_n$, is obtained from Eq. 4. This equation accounts for the longitudinal forces acting on the n-th car, namely the draft-gear forces at the front and rear ends, $F_{dg(n-1)}$ and $F_{dg(n)}$, the motion resistance force, $F_{r(n)}$, and the pneumatic braking force, $F_{b(n)}$. The pneumatic braking model adopted in this study is described in Section 2.2. Similarly, the acceleration of the last freight car, $\ddot{x}_f$, is calculated using Eq. 5.

$$\ddot{x}_n = \frac{-F_{r(n)} - F_{bn} + F_{dg(n-1)} - F_{dg(n)}}{m_n}, \tag{4}$$

$$\ddot{x}_f = \frac{-F_{r(f)} - F_{b(f)} + F_{dg(f-1)}}{m_f}. \tag{5}$$

Once the instantaneous accelerations, $\ddot{x}_n$ [m/s^2], are computed, they are numerically integrated using the Matlab/Simulink™ ODE1 solver to obtain the velocity, $\dot{x}_n$ [m/s], and displacement, x_n [m], of each car. The use of ODE1 is justified by the small time step, $\Delta t \leq 0.0005$ s, required by the brake pipe airflow simulation, and its accuracy has been shown to be comparable to higher-order solvers, such as ODE4 and ODE8 (Eckert et al., 2019).

2.1. Draft gear model

The draft gear forces, denoted by $F_{dg(n)}$, are determined using the model proposed by Wu et al. (2017). This model is coupled with the dissipated-energy approximation method Eckert et al. (2019, 2023), which is applied during the transition between the loading and unloading phases to improve the computational efficiency of the simulations. The adopted formulation accounts for the main mechanical components of the draft gear and their corresponding geometrical characteristics, as illustrated in Fig. 3. The resulting force–displacement relationships are given by

$$F_{j,i} = \Psi_i F_{sm} - (\Psi_i - 1)\, F_{sr}, \qquad i = 1, \ldots, 4, \tag{6}$$

where, $F_{j,i}$ represents the draft gear force associated with the i-th operating stage, where $i = 1, \ldots, 4$. The terms F_{sm} and F_{sr} denote the main spring force and the release spring force, respectively. The parameter Ψ_i is defined as the force amplification factor, accounting for the effects of wedge geometry and frictional contact within the draft gear mechanism.

Since the mechanical response of the draft gear varies throughout the loading and unloading stages, Ψ_i must be evaluated according to the corresponding operating condition. The force amplification factors Ψ_i are defined as

$$\Psi_1 = \frac{1 + \tan(\beta + \arctan(\mu_3)) \tan(\gamma + \arctan(\mu_1))}{1 - \tan(\alpha + \arctan(\mu_2)) \tan(\gamma + \arctan(\mu_1))}, \tag{7}$$

$$\Psi_2 = \Psi_1 + \frac{2\,[1 - \mu_1 \tan(\gamma)]\, \mu_4\, (\Psi_1 - 1)}{\mu_1 + \tan(\gamma)}, \tag{8}$$

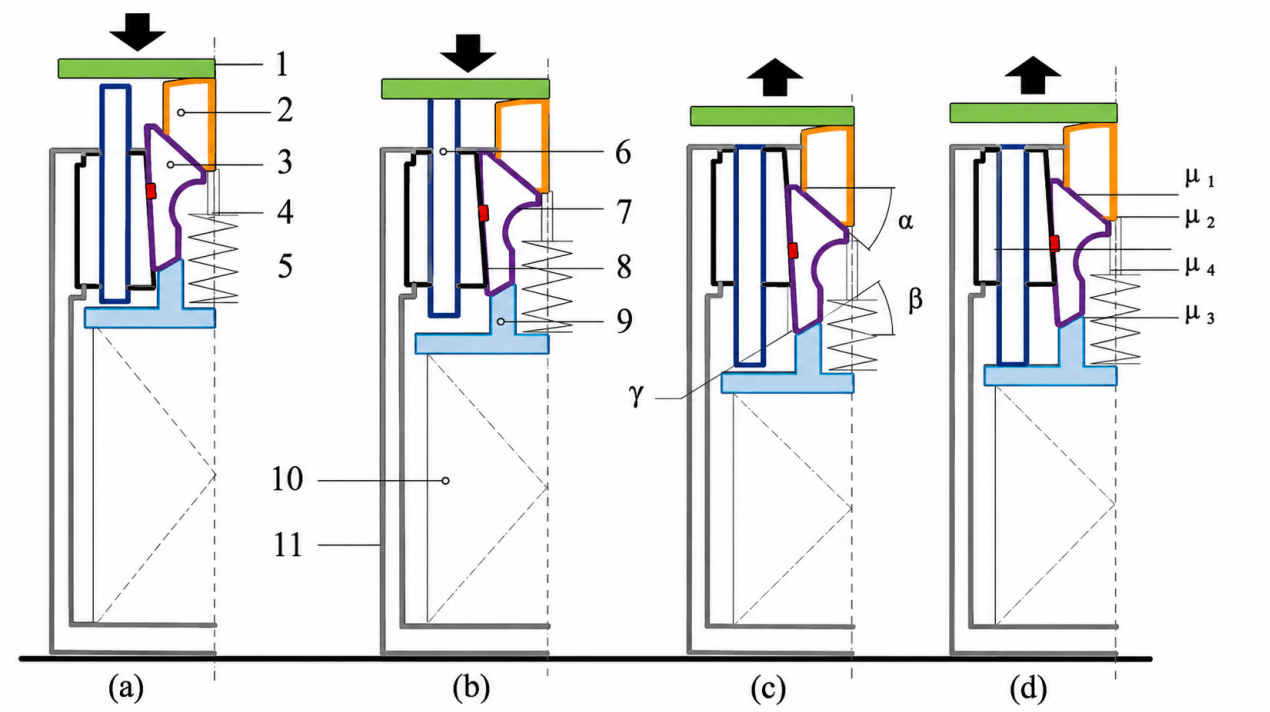


Figure 3: Friction draft gear model: (a) loading stage 1, (b) loading stage 2, (c) unloading stage 1, and (d) unloading stage 2; 1 – follower, 2 – central wedge, 3 – wedge shoe, 4 – release spring, 5 – outer stationary plate, 6 – movable plate, 7 – lubricating metal, 8 – inner stationary plate, 9 – spring seat, 10 – main springs, and 11 – housing.

$$\Psi_3 = \frac{1 + \tan\left(\beta - \arctan\left(\mu_3\right)\right)\tan\left(\gamma - \arctan\left(\mu_1\right)\right)}{1 - \tan\left(\alpha - \arctan\left(\mu_2\right)\right)\tan\left(\gamma - \arctan\left(\mu_1\right)\right)}, \tag{9}$$

and

$$\Psi_4 = \frac{\left[\tan\left(\gamma\right) - \mu_1\right]\Psi_3}{\tan\left(\gamma\right)\left[1 - 2\mu_1\mu_4 + 2\mu_1\mu_4\Psi_3\right] + 2\mu_4\Psi_3 - 2\mu_4 - \mu_1}, \tag{10}$$

where α, β, and γ are the wedge angles indicated in Fig. 3, while μ_1 to μ_4 are the friction coefficients associated with the corresponding contact interfaces.

The friction coefficients between the components, μ_i, are calculated using the exponential formulation presented by Berger (2002) and later applied by Wu et al. (2014) to represent the peak force behavior of friction draft gears. In this formulation, μ_i varies as a function of the relative velocity v_i, allowing the friction behavior to transition between static and dynamic conditions. The parameter σ defines the sensitivity of this variation with respect to velocity. The friction coefficient is then given by

$$\mu_i = \mu_s + \left(\mu_s - \mu_k\right)\exp\left(-\sigma|v_i|\right), \tag{11}$$

where μ_s and μ_k are the static and dynamic friction coefficients, respectively, v_i is the relative velocity at the corresponding contact interface, and σ controls how rapidly μ_i changes with variations in v_i.

After defining the friction coefficients, the relative velocities at the contact interfaces are obtained from the follower velocity of the draft gear. These velocities are required because the friction coefficients μ_i depend on the relative motion between the contacting components. Based on the geometry of the mechanism, the relative velocities v_1, v_2, v_3, and v_4 are expressed as functions of the wedge angles α, β, and γ, and of the follower velocity v_f, as follows:

$$v_1 = \frac{\cos(\alpha)}{\cos(\alpha + \gamma)}v_f, \tag{12}$$

$$v_2 = \frac{\sin(\gamma)}{\cos(\alpha + \gamma)}v_f, \tag{13}$$

$$v_3 = \frac{\cos(\alpha)\sin(\gamma)}{\cos(\alpha + \gamma)\cos(\beta)}v_f, \tag{14}$$

$$v_4(j) = \begin{cases} \dfrac{\cos(\alpha)\sin(\gamma)}{\cos(\alpha+\gamma)\cos(\beta)} v_f, & j = 4, \\ v_f, & j = 2, \\ 0, & j = 1 \text{ or } j = 3. \end{cases} \tag{15}$$

It is worth noting that the four values of the index i in Eqs. (6) and (11) correspond to the four contact interactions considered in the draft gear working process, as illustrated in Fig. 3. The first and second stages are associated with loading, whereas the third and fourth stages describe unloading. Specifically, the first loading stage occurs before the follower comes into contact with the movable plates, while the second loading stage corresponds to the downward motion of these plates. During unloading, the first stage takes place before the spring seat contacts the movable plates, whereas the second unloading stage corresponds to their upward motion. In the present simulations, the model parameters were defined based on the MARK50 draft gear, following the methodology reported in Eckert et al. (2023).

The transition between loading and unloading conditions was treated using the dissipated-energy approximation approach reported in Eckert et al. (2019, 2023). With this procedure, the coupler force is selected according to the current contact condition and the transition state, as expressed by

$$F_c = \begin{cases} 0, & \text{within couplers slack}, \\ \dfrac{F_c(t-dt) + F_j(t)}{2}, & \text{for a loading–unloading transition}, \\ F_j(t), & \text{for all other cases}. \end{cases} \tag{16}$$

where F_c represents the coupler force evaluated at the current time instant. The variable t denotes time, while dt corresponds to the numerical time increment adopted in the simulation. The term $F_j(t)$ is the draft gear force computed from the force–displacement formulation described above. Further information on the draft gear formulation and its implementation is available in Wu et al. (2017) and Eckert et al. (2019, 2023).

2.2. Pneumatic Brake System

The pneumatic brake system is commonly classified into two main configurations: the automatic pneumatic brake system and the electro-pneumatic brake system. In the automatic pneumatic brake system, braking is commanded by pressure variations in the brake pipe, which links the brake components distributed along the train consist. When a pressure reduction occurs in the brake pipe, the control valve, such as the ABDX or DB-60 valves commonly used in railway applications, detects this change and initiates the brake application by connecting the wagon reservoir to the brake cylinders.

Brake release is achieved by increasing the pressure in the brake pipe. This pressure rise changes the operating condition of the control valve, allowing the brake cylinders to exhaust to the atmosphere while the wagon reservoirs are recharged through the brake pipe. One of the main limitations of this system is the delay in brake command transmission, since the application signal depends on the propagation of the pressure reduction along the pipe. In addition, the system is not well suited for repeated sequences of brake application and release, because the reservoirs in each wagon require time to recover their maximum operating pressure. Mathematical models for this type of brake system are presented by Pugi et al. (2013) and Teodoro et al. (2019).

The electro-pneumatic brake system employs electrical commands to control brake application and release along the train consist. In the present study, the system is represented using an EP60 module, which consists of an independent set of electro-pneumatic valves coupled to a conventional pneumatic brake control valve, such as the ABDX or DB-60. This module governs the pneumatic connections involved in the braking process by linking the emergency reservoir to the corresponding brake cylinder during brake application and by connecting the brake cylinder to the atmosphere during brake release. The electro-pneumatic brake model adopted in this work is shown in Fig. 4 and was implemented based on the brake schematic and formulation proposed by Teodoro et al. (2019).

The brake system is commanded by the driver, who sets the desired pressure through the locomotive control system. Based on this command, electrical signals are transmitted from the locomotive to all wagons through a wired connection between the electro-pneumatic valves, with negligible transmission delay. These signals are interpreted by the EP60 control module, which changes the pneumatic connections in each wagon to apply or release the braking force.

During brake application, the EP60 module connects the emergency reservoir to the brake cylinder, allowing the cylinder to be pressurized until the commanded pressure level is reached. During brake release, the module connects the brake cylinder to the atmosphere, reducing its pressure until it reaches the value specified by the control signal. At the same time, the brake pipe continuously supplies the auxiliary and emergency reservoirs, keeping their pressures close to the maximum operating level. As a result, partial releases and successive brake application–release sequences can be performed without reducing the braking capability of the system.

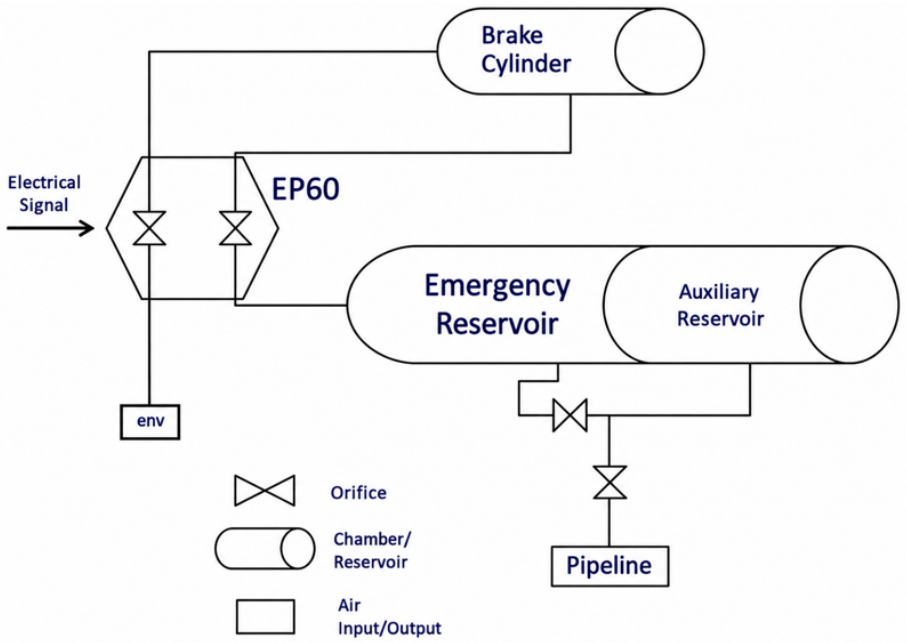


Figure 4: Brake model schematic

The electro-pneumatic brake system is modeled based on the formulation proposed by Pugi et al. (2013). In this approach, pressure losses between connected pneumatic components are represented by orifices, whereas the compressed-air volumes are modeled as chambers, in which the pressure of each component is evaluated. The brake pipe is also discretized as a series of interconnected chambers and orifices. As discussed by Teodoro et al. (2019), this modeling strategy provides satisfactory results while requiring a lower computational cost than more detailed approaches.

3. The SAC Control Algorithm

This section presents the theoretical background behind the proposed control strategy. Subsection 3.1 introduces Markov decision process, the mathematical framework in which reinforcement learning problems are formulated, and defines the policies, returns and value functions used throughout the paper. Subsection 3.2 describes the maximum entropy reinforcement learning framework and the Soft Actor-Critic (SAC) algorithm, which is used to train the agents with continuous actions, as in the pneumatic braking subpolicy. Subsection 3.3 presents the discrete version of SAC, adopted for the meta controller and for the throttle and dynamic braking subpolicies, whose actions are selected from a finite set. Finally, Subsection 3.4 introduces hierarchical reinforcement learning and the meta controller–subpolicy structure, which allows the driving task to be decomposed into distinct operating modes, each handled by a specialized agent.

3.1. Markov Decision Process

Before introducing the control algorithm, it is important to define the problem as a Markov decision process, which is a sequential decision-making process defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$, where, $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $p(s_{t+1} \mid s_t, a_t)$ is the probability of reaching state s_{t+1} after taking action a_t, called the policy, $r(s_t, a_t)$ is the reward received for that transition, and $\gamma \in [0, 1)$ is a discount factor (Sutton & Barto, 2018). The sequence is then: at each time step t, the agent observes the environment state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$ following a policy $\pi(a_t \mid s_t)$. The environment then moves to a new state s_{t+1} and returns a reward $r_t = r(s_t, a_t)$.

In this work, the environment is the multibody model presented in Section 2. The state s_t is defined by a 10 14-dimensional vector that includes the time, position, velocity, locomotive notch point, brake pipe pressure and others that will be described in detail in Section 3.6. After observing the state, the agent selects an action a_t that corresponds to a throttle notch, a dynamic braking notch or a brake pipe pressure command. This process will be explained in Section 3.7. The environment then transitions to a new state s_{t+1} following the equation:

$$s_{t+1} = f(s_t, a_t) \tag{17}$$

where f corresponds to the multibody model dynamics. The agent then receives a reward r_t, described in Section 3.8, which is used to update the policy. This process is repeated until the end of the episode.

The agent's performance is measured by the discounted return, defined as the sum of future rewards from time t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k}. \tag{18}$$

The objective of agent is to find a policy that maximizes the expected return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \, r(s_t, a_t) \right] \tag{19}$$

To evaluate the performance of the policy, many reinforcemnet learning methods try to estimate the value function. The value function $V^\pi(s)$ is the expected return when starting in state s and following π thereafter. Similarly, it is possible to define an action-value function $Q^\pi(s, a)$, which is the expected return when taking action a in state s and following π thereafter (Sutton & Barto, 2018):

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid s_t = s, a_t = a \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k \, r_{t+k} \mid s_t = s, a_t = a \right]. \tag{20}$$

The function Q satisfies the Bellman equation, which links the value of a state–action pair to the immediate reward and the value of the next pair:

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \, \mathbb{E}_{s_{t+1} \sim p,\, a_{t+1} \sim \pi} \left[Q^\pi(s_{t+1}, a_{t+1}) \right]. \tag{21}$$

The optimal policy π^* is the one that achieves the highest Q-value in every state, and its Q-function Q^* satisfies the Bellman optimality equation:

$$Q^*(s_t, a_t) = r(s_t, a_t) + \gamma \, \mathbb{E}_{s_{t+1} \sim p} \left[\max_{a'} Q^*(s_{t+1}, a') \right]. \tag{22}$$

3.2. The Soft Actor-Critic Algorithm

In standard RL, the agent maximizes only the expected return. In the maximum entropy framework (Ziebart et al., 2008; Ziebart, 2010; Haarnoja et al., 2017), The objective is modified to include an entropy term, and is defined as:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \Big(r(s_t, a_t) + \alpha \, \mathcal{H}\big(\pi(\cdot \mid s_t)\big) \Big) \right], \tag{23}$$

where $\mathcal{H}(\pi(\cdot \mid s_t)) = -\mathbb{E}_{a \sim \pi} \left[\log \pi(a \mid s_t) \right]$ is the entropy of the policy and $\alpha > 0$ is the temperature parameter. This objective encourages the agent to act as randomly as possible while still performing the task. That improves exploration, prevents early convergence to poor deterministic behaviors, and lets the policy keep several near-optimal ways of acting (Haarnoja et al., 2017, 2018a).

With the addition of the entropy term, the value functions presented in Section 3.1 modified. The soft Q-function satisfies a modified Bellman equation (Haarnoja et al., 2018a):

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \, \mathbb{E}_{s_{t+1} \sim p} \left[V^\pi(s_{t+1}) \right], \tag{24}$$

where the soft state value function is

$$V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi} \left[Q^\pi(s_t, a_t) - \alpha \log \pi(a_t \mid s_t) \right]. \tag{25}$$

SAC is derived from soft policy iteration, which alternates between two steps. In policy evaluation, the soft Q-function of the current policy is computed by repeatedly applying Eq. (24). In policy improvement, the policy is updated toward the exponential of the soft Q-function, by minimizing the Kullback–Leibler (KL) divergence:

$$\pi_{\text{new}} = \arg\min_{\pi' \in \Pi} \; D_{\text{KL}} \left(\pi'(\cdot \mid s_t) \,\middle\|\, \frac{\exp\left(Q^{\pi_{\text{old}}}(s_t, \cdot)/\alpha \right)}{Z^{\pi_{\text{old}}}(s_t)} \right), \tag{26}$$

where Π is the family of admissible policies (for example, Gaussian distributions) and $Z^{\pi_{\text{old}}}(s_t)$ is a normalization term. Haarnoja et al. (2018a) showed that, in the tabular case, repeating these two steps converges to the optimal policy within Π. For large or continuous state spaces, both steps are approximated with neural networks and stochastic gradient descent, which gives the practical SAC algorithm described below.

SAC learns two soft Q-functions, Q_{θ_1} and Q_{θ_2}, each represented by a neural network with parameters θ_i. Using the minimum of the two estimates when computing targets reduces the overestimation bias common in value-based methods. This technique is known as clipped double Q-learning (Fujimoto et al., 2018), and it builds on the ideas of double Q-learning (van Hasselt, 2010). Each critic is trained on transitions (s_t, a_t, r_t, s_{t+1}) sampled from a replay buffer $\mathcal{D}$ that stores past experience (Lin, 1992; Mnih et al., 2015), by minimizing the soft Bellman residual:

$$J_Q(\theta_i) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim\mathcal{D}}\left[\frac{1}{2}\Big(Q_{\theta_i}(s_t,a_t) - y_t\Big)^2\right], \quad i = 1, 2, \tag{27}$$

with the target

$$y_t = r_t + \gamma\left(\min_{j=1,2} Q_{\bar{\theta}_j}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1} \mid s_{t+1})\right), \quad a_{t+1} \sim \pi_\phi(\cdot \mid s_{t+1}). \tag{28}$$

Here $Q_{\bar{\theta}_j}$ are target networks, whose parameters slowly track the critic parameters through Polyak averaging, $\bar{\theta}_j \leftarrow \tau_p\theta_j + (1-\tau_p)\bar{\theta}_j$, with $\tau_p \ll 1$. This stabilizes the targets during training (Mnih et al., 2015; Lillicrap et al., 2016).

For continuous actions, the policy π_ϕ is a Gaussian distribution whose mean $\mu_\phi(s_t)$ and standard deviation $\sigma_\phi(s_t)$ are produced by a neural network with parameters ϕ. Since physical actions are bounded, the Gaussian sample is passed through a hyperbolic tangent so that the action lies in $(-1, 1)$ (Haarnoja et al., 2018a). The action is written using the reparameterization trick (Kingma & Welling, 2014):

$$a_t = f_\phi(\xi_t; s_t) = \tanh\big(\mu_\phi(s_t) + \sigma_\phi(s_t) \odot \xi_t\big), \qquad \xi_t \sim \mathcal{N}(0, I). \tag{29}$$

Writing the action this way makes it a differentiable function of ϕ, so gradients can flow from the critic into the policy network. The policy parameters are updated by minimizing the expected KL divergence of Eq. (26), which is equivalent to the loss

$$J_\pi(\phi) = \mathbb{E}_{s_t\sim\mathcal{D},\,\xi_t\sim\mathcal{N}}\left[\alpha \log \pi_\phi\big(f_\phi(\xi_t; s_t) \mid s_t\big) - \min_{j=1,2} Q_{\theta_j}\big(s_t, f_\phi(\xi_t; s_t)\big)\right]. \tag{30}$$

Because of the tanh transformation, the log-probability in Eq. (30) has to include a change-of-variables correction,

$$\log \pi_\phi(a_t \mid s_t) = \log \mathcal{N}(u_t; \mu_\phi, \sigma_\phi) - \sum_i \log\big(1 - \tanh^2(u_{t,i})\big), \tag{31}$$

where u_t is the Gaussian sample before squashing (Haarnoja et al., 2018a). The bounded action is then scaled linearly to the physical range of the command. In this work, that is the brake pipe pressure between 0 and 90 psi.

The temperature α strongly affects performance, and its best value depends on the task and the reward scale. Haarnoja et al. (2018b) proposed adjusting it automatically by constraining the average entropy of the policy to stay above a target value $\bar{\mathcal{H}}$. This leads to the loss

$$J(\alpha) = \mathbb{E}_{s_t\sim\mathcal{D},\,a_t\sim\pi_\phi}\left[-\alpha \log \pi_\phi(a_t \mid s_t) - \alpha\bar{\mathcal{H}}\right]. \tag{32}$$

When the policy's entropy falls below the target, α increases, encouraging exploration; when it is above the target, α decreases. For continuous actions, the heuristic $\bar{\mathcal{H}} = -\dim(\mathcal{A})$ is commonly used (Haarnoja et al., 2018b), which gives $\bar{\mathcal{H}} = -1$ for the single-dimensional pressure command considered here.

Training alternates between collecting experience and updating the networks. At each environment step, an action is sampled from the current policy, the resulting transition is stored in $\mathcal{D}$, and a mini-batch is drawn from $\mathcal{D}$ to update the critics (Eq. (27)), the actor (Eq. (30)), the temperature (Eq. (32)), and the target networks. Because the updates use data from the replay buffer rather than only from the current policy, SAC reuses experience efficiently. That matters here, since each episode requires simulating the full multibody train model.

3.3. Soft Actor-Critic for Discrete Actions

The SAC algorithm presented in Section 3.2 was formulated for continuous action spaces. Its policy update relies on the reparameterization trick of Eq. (29), which requires the action to be a differentiable function of the policy parameters. This is not possible when the action is chosen from a finite set, as in the selection of a gear point or of the active subpolicy. Christodoulou (2019) proposed an adaptation of SAC to discrete action spaces that keeps the maximum entropy objective of Eq. (23) and the overall actor-critic structure, but changes how the policy and the Q-functions are represented. This version, referred to here as discrete SAC, is used for the meta controller and for the throttle and dynamic braking subpolicies.

Policy and critic representation

Let the action space be a finite set $\mathcal{A} = \{a^{(1)}, \dots, a^{(|\mathcal{A}|)}\}$. Instead of producing the parameters of a Gaussian distribution, the policy network outputs one logit per action, which a softmax function turns into a probability vector:

$$\pi_\phi(s_t) = \mathrm{softmax}\big(z_\phi(s_t)\big) \in [0,1]^{|\mathcal{A}|}, \qquad \sum_{k=1}^{|\mathcal{A}|} \pi_\phi\big(a^{(k)} \mid s_t\big) = 1, \tag{33}$$

where $z_\phi(s_t) \in \mathbb{R}^{|\mathcal{A}|}$ is the vector of logits. Similarly, instead of taking the action as an input, each critic receives only the state and outputs the Q-value of every action at once, $Q_{\theta_i}(s_t) \in \mathbb{R}^{|\mathcal{A}|}$, in the same way as the Deep Q-Network (Mnih et al., 2015). Twin critics, target networks updated by Polyak averaging, and a replay buffer are kept exactly as in the continuous case (Christodoulou, 2019).

Exact expectations over actions

The main advantage of a finite action space is that expectations over the policy no longer need to be estimated by sampling: they can be computed exactly by summing over all actions, weighted by their probabilities. The soft state-value function of Eq. (25) then becomes

$$V(s_t) = \pi_\phi(s_t)^\top \Big[Q(s_t) - \alpha \log \pi_\phi(s_t) \Big], \tag{34}$$

where the logarithm is applied element-wise (Christodoulou, 2019). Using Eq. (34), the critic target of Eq. (28) is rewritten as

$$y_t = r_t + \gamma\, \pi_\phi(s_{t+1})^\top \Big[\min_{j=1,2} Q_{\bar{\theta}_j}(s_{t+1}) - \alpha \log \pi_\phi(s_{t+1}) \Big], \tag{35}$$

where the minimum is taken element-wise over the two target critics. Each critic is trained by minimizing the same squared error of Eq. (27), using the Q-value of the action actually taken, $Q_{\theta_i}(s_t)[a_t]$. Compared with the continuous case, the target no longer depends on a sampled next action, which reduces its variance.

Policy update

Since the expectation over actions can be computed exactly, the policy loss of Eq. (30) no longer requires the reparameterization trick. It becomes

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} \Big[\pi_\phi(s_t)^\top \Big(\alpha \log \pi_\phi(s_t) - \min_{j=1,2} Q_{\theta_j}(s_t) \Big) \Big], \tag{36}$$

which is differentiable with respect to ϕ through the action probabilities themselves (Christodoulou, 2019). Minimizing Eq. (36) increases the probability of actions with high Q-values while the entropy term prevents the distribution from collapsing too quickly onto a single action.

Temperature adjustment

The temperature is adjusted automatically as in Eq. (32), again replacing the sampled expectation by an exact one:

$$J(\alpha) = \mathbb{E}_{s_t \sim \mathcal{D}} \Big[\pi_\phi(s_t)^\top \Big(-\alpha \log \pi_\phi(s_t) - \alpha \bar{\mathcal{H}} \Big) \Big]. \tag{37}$$

The heuristic $\bar{\mathcal{H}} = -\dim(\mathcal{A})$ used for continuous actions does not apply here, since the entropy of a categorical distribution is always non-negative and is bounded above by $\log|\mathcal{A}|$, the entropy of the uniform distribution. Christodoulou (2019) therefore sets the target as a fraction of this maximum, $\bar{\mathcal{H}} = 0.98 \log|\mathcal{A}|$. Table 2 lists the size of the action space of each discrete agent in this work.

Table 2: Action spaces of the agents trained with discrete SAC.

Agent	Actions	$\lvert\mathcal{A}\rvert$
Meta controller	throttle, dynamic braking, pneumatic braking	3
Throttle subpolicy	gear points 0 to 8	9
Dynamic braking subpolicy	gear points -1 to -4	4

Action selection

During training, actions are sampled from the categorical distribution $\pi_\phi(s_t)$, so that the agent keeps exploring. After training, the policy can either continue to sample actions or select the most probable one, $a_t = \arg\max_k \pi_\phi\big(a^{(k)} \mid s_t\big)$, which gives a deterministic controller.

3.4. Hierarchical Reinforcement Learning

Many control problems involve decisions at different levels of abstraction. Driving a train, for example, involves deciding whether to accelerate, hold speed with electrical braking, or apply the air brakes, and then deciding how intensely to do so. A single, flat policy that handles all of these decisions at once must learn a large mapping from states to actions, which can make training slow and the resulting behavior hard to interpret. Hierarchical reinforcement learning (HRL) addresses this by decomposing the decision process into levels, where higher levels choose among sub-tasks or behaviors and lower levels execute them (Barto & Mahadevan, 2003; Pateria et al., 2021).

Classical frameworks

Several formulations of HRL have been proposed. In feudal reinforcement learning (Dayan & Hinton, 1993), managers at higher levels assign sub-goals to workers at lower levels. The hierarchies of abstract machines (Parr & Russell, 1998) constrain the policy with partially specified finite-state controllers, while the MAXQ method (Dietterich, 2000) decomposes the value function of the overall task into the value functions of its sub-tasks. The most widely adopted formulation is the options framework (Sutton et al., 1999). An option ω is a temporally extended action defined by the tuple $(\mathcal{I}_\omega, \pi_\omega, \beta_\omega)$, where $\mathcal{I}_\omega \subseteq \mathcal{S}$ is the set of states in which the option can start, π_ω is the policy followed while the option is active, and $\beta_\omega(s) \in [0,1]$ is the probability that the option terminates in state s. A higher-level policy over options, $\pi_\Omega(\omega \mid s)$, selects which option to execute; once selected, the option controls the agent until it terminates. Because options can last a variable number of steps, the decision process seen by the higher level is a semi-Markov decision process (SMDP), and its Q-function satisfies

$$Q_\Omega(s_t, \omega) = \mathbb{E}\left[\sum_{k=0}^{\kappa-1} \gamma^k r_{t+k} + \gamma^\kappa \max_{\omega'} Q_\Omega(s_{t+\kappa}, \omega')\right], \tag{38}$$

where κ is the random number of steps for which option ω remains active (Sutton et al., 1999). When every option lasts exactly one step ($\beta_\omega \equiv 1$), Eq. (38) reduces to the Bellman optimality equation of an ordinary MDP, Eq. (**??**).

Deep hierarchical architectures

With the development of deep RL, these ideas were combined with neural network function approximation. Kulkarni et al. (2016) proposed a two-level architecture in which a meta controller selects a goal and a lower-level controller chooses primitive actions to reach it, both trained with deep Q-learning. Other deep HRL methods learn the options themselves end to end, as in the option-critic architecture (Bacon et al., 2017), or use feudal structures with learned sub-goals (Vezhnevets et al., 2017; Nachum et al., 2018). A common feature of these approaches is that each level can be trained with a standard RL algorithm, treating the levels below it as part of its environment.

Hierarchy and hybrid action spaces

Hierarchical structures are also a natural way to handle hybrid, or parameterized, action spaces, in which the agent must first choose a discrete action type and then a set of parameters specific to that type (Masson et al., 2016; Hausknecht & Stone, 2016). In such problems, the parameters of different action types can differ in number, range, and even in nature, being discrete for some types and continuous for others. Assigning the choice of action type to a higher-level policy, and the choice of parameters to a separate lower-level policy for each type, lets each policy work in a simple, homogeneous action space and be trained with the algorithm best suited to it.

Hierarchical formulation adopted in this work

The train driving problem considered here has exactly this structure. The locomotive can be in one of three operating modes, namely throttle, dynamic braking, and pneumatic braking, and each mode has its own command: a discrete gear point between 0 and 8, a discrete gear point between -1 and -4, or a continuous brake pipe pressure between 0 and 90 psi, respectively. Following the meta controller structure of Kulkarni et al. (2016), the control policy is split into two levels. At the upper level, a meta policy $\pi_M(m_t \mid s_t)$ selects the operating mode $m_t \in \mathcal{M} = \{\text{thr}, \text{db}, \text{pb}\}$. At the lower level, the selected subpolicy $\pi_{m_t}(u_t \mid s_t)$ chooses the command u_t within its own action space $\mathcal{U}_{m_t}$. The command is then mapped to the locomotive inputs, gear point p_t and brake pipe pressure $p_{f,t}$, through a mode-specific function

$$(p_t,\, p_{f,t}) = g_{m_t}(u_t) = \begin{cases} (u_t,\, 0), & m_t = \text{thr}, \quad u_t \in \{0, 1, \ldots, 8\}, \\ (u_t,\, 0), & m_t = \text{db}, \quad u_t \in \{-1, -2, -3, -4\}, \\ (-4,\, u_t), & m_t = \text{pb}, \quad u_t \in [0, 90] \text{ psi}. \end{cases} \tag{39}$$

This structure also guarantees by construction that traction and pneumatic braking are never applied at the same time. Because the meta controller and the throttle and dynamic braking subpolicies have discrete action spaces, they are trained with discrete SAC (Section 3.3), whereas the pneumatic braking subpolicy, whose action is continuous, is trained with the original SAC (Section 3.2).

In this work, the meta controller selects a mode at every time step. In terms of the options framework, each subpolicy behaves as an option that terminates after a single step ($\beta_\omega \equiv 1$), so the meta controller faces an ordinary MDP and no semi-Markov formulation is needed. The benefit of the hierarchy therefore comes from the decomposition of the action space rather than from temporal abstraction: each agent solves a simpler problem, and the sequence of selected modes along the route remains directly interpretable as a driving strategy.

The details of the state, the reward, and the training scheme of each agent are given in Sections 3.5 and **??**.

3.5. Proposed Architecture

3.6. State Space

$$s_t = (t, x, V, p, BP, ACC, A, A_L, C, C_L, I, I_L, D, D_L) \tag{40}$$

where t is the time, x is the position of the train along the track, V is the train speed, p is the locomotive's notch position, BP is the brake pipe pressure, ACC is the train acceleration, A is the altitude of the track at the train's position, A_L is the altitude of the track 100 m ahead of the train, C is the curvature of the track at the train's position, C_L is the curvature of the track 100 m ahead of the train, I is a binary variable indicating whether the train is in an uphill section, and I_L is a binary variable indicating whether the train will be in an uphill section 100 m ahead of the train, D is a binary variable indicating whether the train is in a downhill section, and D_L is a binary variable indicating whether the train will be in a downhill section 100 m ahead of the train. These variables will be discussed in more detail in Section 3.6.

3.7. Action Space

3.8. Reward Function

4. Methodology

In the next sections, we will present the approach taken to design an efficient and safe train control system using a Hybrid Soft Actor-Critic (SAC) model described in Delalleau et al. (2019). The goal is to optimize energy consumption and travel time while maintaining safety. The model dynamically adjusts the locomotive's throttle and brake pressure to achieve this balance.

4.1. Model Structure

In reinforcement learning, an agent learns the optimal control strategies through continuous interaction with its environment, aiming to maximize the cumulative reward over time. At each time step t, the agent selects an action $a_t \in \mathcal{A}$, transitioning the system from the current state s_t to a new state s_{t+1}, with $s_t, s_{t+1} \in \mathcal{S}$. This transition yields an immediate reward r_t, which informs the agent about the quality of its chosen action within the context of its long-term goal.

The actions chosen by the agent must adhere to the constraints defined by the problem domain, such as physical limitations, safety requirements, or system-specific regulations—an example being a restriction on the maximum allowable velocity. These constraints ensure the agent's behavior remains realistic and applicable to real-world scenarios.

Figure (INSERT FIGURE HERE) illustrates the general structure of the reinforcement learning model, depicting the cyclic process of decision-making, state transition, and reward evaluation. This diagram provides an overview of how the agent interacts with the environment, continually refining its policy to achieve optimal performance.

4.2. Model Architecture

Optimization equations:

$$\overline{Q_{target}} = r_t + \gamma \mathbb{E}_{s_{t+1}} \left(\sum_{a_d} \pi\left(a_d|s_{t+1}\right) \min\left(Q_1, Q_2\right) - \alpha_d \log\left(\pi\left(a_d|s_{t+1}\right)\right) - \alpha_c \sum_{a_d} \pi\left(a_d|s_{t+1}\right) \log\left(\pi\left(a_c|s_{t+1}, a_d\right)\right) \right) \tag{41}$$

$$J_\theta^i = \frac{1}{N} \sum_{i=1}^{N} \left(Q_j^i - \overline{Q_{target}^i} \right)^2 \tag{42}$$

for $j = 1, 2$. The combined loss will be the mean of individual Q-network losses:

$$J_\theta^C = \frac{J_\theta^1 + J_\theta^2}{2} \tag{43}$$

The policy loss was divided into two parts, the discrete and the continuous, that were computed as:

$$J_{\pi}^{d} = \mathbb{E}_{s_t}\left(\sum_{a_d} \pi\left(a_d|s_t\right) \cdot \left[\alpha_d \log\left(\pi\left(a_d|s_t\right)\right) - \min\left(Q_1, Q_2\right)\right]\right) \tag{44}$$

$$J_{\pi}^{c} = \mathbb{E}_{s_t}\left(\sum_{a_d} \pi\left(a_d|s_t\right) \cdot \left[\alpha_c \sum_{a_d} \pi\left(a_d|s_t\right) \log\left(\pi\left(a_d|s_t\right)\right) - \min\left(Q_1, Q_2\right)\right]\right) \tag{45}$$

The final loss is computed as the sum of the continuous and discrete parts:

$$J_{pi}^{C} = J_{\pi}^{d} + J_{\pi}^{c} \tag{46}$$

The loss for the parameters α_c and α_d is computed as follows:

$$J_{\alpha_d} = \mathbb{E}\left(-\log\left(\alpha_d\right) \sum_{a_d} \pi\left(a_d|s_t\right) \cdot \left[\log\left(\pi\left(a_d|s_t\right)\right) + \alpha_{target}^{d}\right]\right) \tag{47}$$

$$J_{\alpha_c} = \mathbb{E}\left(-\log\left(\alpha_c\right) \sum_{a_d} \pi\left(a_d|s_t\right) \cdot \left[\sum_{a_d} \pi\left(a_d|s_t\right) \log\left(\pi\left(a_c|s_t, a_d\right)\right) + \alpha_{target}^{c}\right]\right) \tag{48}$$

The two target networks were updated using:

$$\overline{\theta_{t+1}^{j}} = \tau\theta_{t}^{j} + \left(1-\tau\right)\overline{\theta_{t}^{j}} \tag{49}$$

for $j = 1, 2$

5. Results

6. Conclusion

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was conducted with scholarships and financial support from Research Development Foundation - FUNDEP Rota 2030/Line V, from São Paulo Research Foundation - FAPESP (2021/14026-8), from Brazilian National Council for Scientific and Technological Development - CNPq (308389/2022-0 and 407874/2022-4) the University of Campinas - UNICAMP.

References

Bacon, P.-L., Harb, J., & Precup, D. (2017). The option-critic architecture. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence* (pp. 1726–1734).

Bai, B., Xiao, Z., Wang, Q., Sun, P., & Feng, X. (2020). Multi-objective trajectory optimization for freight trains based on quadratic programming. *Transportation Research Record*, *2674*, 466–477. doi:10.1177/0361198120937307.

Barto, A. G., & Mahadevan, S. (2003). Recent advances in hierarchical reinforcement learning. *Discrete Event Dynamic Systems*, *13*, 341–379.

Berger, E. (2002). Friction modeling for dynamic system simulation. *Appl. Mech. Rev.*, *55*, 535–577.

Chen, X., Guo, X., Meng, J., Xu, R., Li, S., & Li, D. (2023). Research on ato control method for urban rail based on deep reinforcement learning. *IEEE Access*, *11*, 5919–5928.

Cheng, L., Cao, J., Yang, X., Wang, W., & Zhou, Z. (2024). A train trajectory optimization method based on the safety reinforcement learning with a relaxed dynamic reward. *Discover Applied Sciences*, *6*, 469.

Christodoulou, P. (2019). Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*, .

Cole, C. et al. (2006). Longitudinal train dynamics. *Handbook of railway vehicle dynamics*, (pp. 239–278).

Dayan, P., & Hinton, G. E. (1993). Feudal reinforcement learning. In *Advances in Neural Information Processing Systems 5* (pp. 271–278).

Delalleau, O., Peter, M., Alonso, E., & Logut, A. (2019). Discrete and continuous action representation for practical rl in video games. URL: https://arxiv.org/abs/1912.11077. arXiv:1912.11077.

Dietterich, T. G. (2000). Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research*, *13*, 227–303.

Eckert, J. J., Pavani Teodoro, Í., Valente Lopes, M., Wu, Q., & Santos, A. A. (2025). Multi-objective optimization of electro-pneumatic braking process with fuzzy logic control for heavy haul railway applications. *International Journal of Rail Transportation*, *13*, 332–351.

Eckert, J. J., Ramos, P. G., de Oliveira Junior, A. J. S., da Silva Martins, T., & Kurka, P. R. G. (2019). A dissipated energy model of shock evolution in the simulation of the dynamics of dgm's of railway compositions. *Mechanism and Machine Theory*, *134*, 365–375. doi:10.1016/j.mechmachtheory.2018.12.034.

Eckert, J. J., Teodoro, Í. P., Teixeira, L. H., Martins, T. S., Kurka, P. R., & Santos, A. A. (2023). A fast simulation approach to assess draft gear loads for heavy haul trains during braking. *Mechanics Based Design of Structures and Machines*, *51*, 1606–1625. doi:10.1080/15397734.2021.1875233.

Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning* (pp. 1587–1596). volume 80 of *Proceedings of Machine Learning Research*.

Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017). Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1352–1361). volume 70 of *Proceedings of Machine Learning Research*.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018a). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning* (pp. 1861–1870). volume 80 of *Proceedings of Machine Learning Research*.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., & Levine, S. (2018b). Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, .

van Hasselt, H. (2010). Double Q-learning. In *Advances in Neural Information Processing Systems 23* (pp. 2613–2621).

Hausknecht, M., & Stone, P. (2016). Deep reinforcement learning in parameterized action space. In *International Conference on Learning Representations (ICLR)*.

Howlett, P. (2016). A new look at the rate of change of energy consumption with respect to journey time on an optimal train journey. *Transportation Research Part B: Methodological*, *94*, 387–408. doi:10.1016/j.trb.2016.10.004.

Huang, Y., Lai, W., Chen, D., Lin, G., & Yin, J. (2025). Enhanced intelligent train operation algorithms for metro train based on expert system and deep reinforcement learning. *PLoS One*, *20*, e0323478.

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*.

Kulkarni, T. D., Narasimhan, K. R., Saeedi, A., & Tenenbaum, J. B. (2016). Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *Advances in Neural Information Processing Systems 29* (pp. 3675–3683).

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, *8*, 293–321.

Lin, X., Wang, Q., Wang, P., Sun, P., & Feng, X. (2017). The energy-efficient operation problem of a freight train considering long-distance steep downhill sections. *Energies*, *10*, 794. doi:10.3390/en10060794.

Masson, W., Ranchod, P., & Konidaris, G. (2016). Reinforcement learning with parameterized actions. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence* (pp. 1934–1940).

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*, 529–533.

Nachum, O., Gu, S., Lee, H., & Levine, S. (2018). Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems 31*.

Oprea, R. A., Cruceanu, C., & Spiroiu, M. A. (2013). Alternative friction models for braking train dynamics. *Vehicle System Dynamics*, *51*, 460–480. doi:10.1080/00423114.2012.744459.

Parr, R., & Russell, S. (1998). Reinforcement learning with hierarchies of machines. In *Advances in Neural Information Processing Systems 10* (pp. 1043–1049).

Pateria, S., Subagdja, B., Tan, A.-h., & Quek, C. (2021). Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys*, *54*, 1–35.

Pugi, L., Malvezzi, M., Papini, S., & Vettori, G. (2013). Design and preliminary validation of a tool for the simulation of train braking performance. *Journal of Modern Transportation*, *21*, 247–257. doi:10.1007/s40534-013-0027-6.

Shang, M., Zhou, Y., & Fujita, H. (2021). Deep reinforcement learning with reference system to handle constraints for energy-efficient train control. *Information Sciences*, *570*, 708–721.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. (2nd ed.). The MIT Press.

Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, *112*, 181–211.

Teodoro, Í. P., Ribeiro, D. F., Botari, T., Martins, T. S., & Santos, A. A. (2019). Fast simulation of railway pneumatic brake systems. *Proceedings of the Institution of Mechanical Engineers, Part F: Journal of Rail and Rapid Transit*, *233*, 420–430. doi:10.1177/0954409718796903.

Vezhnevets, A. S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., & Kavukcuoglu, K. (2017). FeUdal networks for hierarchical reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 3540–3549). volume 70 of *Proceedings of Machine Learning Research*.

Wagner, S., Cole, C., & Spiryagin, M. (2021). A review on design and testing methodologies of modern freight train draft gear system. *Railway Engineering Science*, *29*, 127–151. doi:10.1007/s40534-021-00237-y.

Wang, X., Li, S., Tang, T., Wang, X., & Xun, J. (2019). Intelligent operation of heavy haul train with data imbalance: A machine learning method. *Knowledge-Based Systems*, *163*, 36–50. doi:10.1016/j.knosys.2018.08.015.

Wei, G., Zhu, S., Wang, Y., Chen, W., Lu, S., Sha, S., & Li, L. (2022). Energy-efficient automatic train operation for high-speed railways: Considering discrete notches and neutral sections. *Transportation Research Part C: Emerging Technologies*, *145*, 103884.

Wei, W., Hu, Y., Wu, Q., Zhao, X., Zhang, J., & Zhang, Y. (2017). An air brake model for longitudinal train dynamics studies. *Vehicle System Dynamics*, *55*, 517–533. doi:10.1080/00423114.2016.1254261.

Wu, Q., Cole, C., Luo, S., & Spiryagin, M. (2014). A review of dynamics modelling of friction draft gear. *Vehicle System Dynamics*, *52*, 733–758. doi:10.1080/00423114.2014.894199.

Wu, Q., Cole, C., Spiryagin, M., Chang, C., Wei, W., Ursulyak, L., Shvets, A., Murtaza, M. A., Mirza, I. M., Zhelieznov, K., Mohammadi, S., Serajian, H., Schick, B., Berg, M., Sharma, R. C., Aboubakr, A., Sharma, S. K., Melzi, S., Gialleonardo, E. D., Bosso, N., Zampieri, N., Magelli, M., Ion, C. C., Routcliffe, I., Pudovikov, O., Menaker, G., Mo, J., Luo, S., Ghafourian, A., Serajian, R., Santos, A. A., Teodoro, I. P., Eckert, J. J., Pugi, L., Shabana, A., & Cantone, L. (2023). Freight train air brake models. *International Journal of Rail Transportation*, *11*, 1–49. doi:10.1080/23248378.2021.2006808.

Wu, Q., Cole, C., Spiryagin, M., & McSweeney, T. (2018). Parallel multiobjective optimisations of draft gear designs. *Proceedings of the Institution of Mechanical Engineers, Part F: Journal of Rail and Rapid Transit*, *232*, 744–758. doi:10.1177/0954409717690981.

Wu, Q., Luo, S., Qu, T., & Yang, X. (2017). Comparisons of draft gear damping mechanisms. *Vehicle System Dynamics*, *55*, 501–516. doi:10.1080/00423114.2016.1252049.

Yadav, O. P., & Vyas, N. S. (2023). The influence of aar coupler features on estimation of in-train forces. *Railway Engineering Science*, (pp. 1–19). doi:10.1007/s40534-022-00297-8.

Yi, L., Zhang, D., Li, W., Duan, R., Jiang, P., & Liu, B. (2022). Research on multi-objective optimization of freight train operation process based on improved bald eagle search algorithm. *Journal of Computers*, *33*, 135–150. doi:10.53106/199115992022103305012.

Zhang, L., & Zhuan, X. (2013). Optimal operation of heavy-haul trains equipped with electronically controlled pneumatic brake systems using model predictive control methodology. *IEEE Transactions on Control Systems Technology*, *22*, 13–22. doi:10.1109/TCST.2013.2238235.

Zhou, K., Song, S., Xue, A., You, K., & Wu, H. (2020). Smart train operation algorithms based on expert knowledge and reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *52*, 716–727.

Ziebart, B. D. (2010). *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. Ph.D. thesis Carnegie Mellon University.

Ziebart, B. D., Maas, A., Bagnell, J. A., & Dey, A. K. (2008). Maximum entropy inverse reinforcement learning. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence* (pp. 1433–1438).